The analogous with randomized algorithms is **local optimization**: we can have sub-heuristics performing a local optimization - for example, trying to move vertices around, one by one, within a short distance from the randomly-assigned initial position, and check if the crossing number improves.

This algorithm is called *hill climbing* - or, in our case, since we try to minimize a function instead of maximizing it, we can call it **hill descent**.

Figure 16.12 visualizes the analogy: in our case despite the nudge the marble only travels a short distance (well, it’s a muddy track, isn’t it?), but we might still get some improvement. What we really do is exploring the cost function in a small area around each solution, and if we find a position for which the cost is lower, we move our “marble” there.

If you look closely at figure 16.12 you can notice that, in this case, the shape of the cost function does matter: while with a differentiable, bowl-shaped function we always get an improvement, with a step function (like our example, “minimum number of intersections”), sometimes the marbles are stuck in local minima, sometimes they are on large plateaus, and so moving them around won’t get us anywhere better.